# Building a RAG Pipeline

## Using LangChain and FAISS

Mondweep Chakravorty

Knowledge Sharing Session

December 2025

# Session Agenda

- **2 mins** - Introduction
- **5 mins** - Presentation
- **3 mins** - Code Demo
- **30 mins** - Q&A

**Target Audience**

Beginners with limited understanding of Generative AI

# Retrieval Augmented Generation

A technique that enhances LLMs by giving them access to external knowledge sources

**The Problem RAG Solves:**

- LLMs have knowledge cutoff dates
- They can "hallucinate" (make things up)
- They don't know your private/company data

# RAG vs Standard LLM

**Without RAG**

**User:** "What is our company's leave policy?"

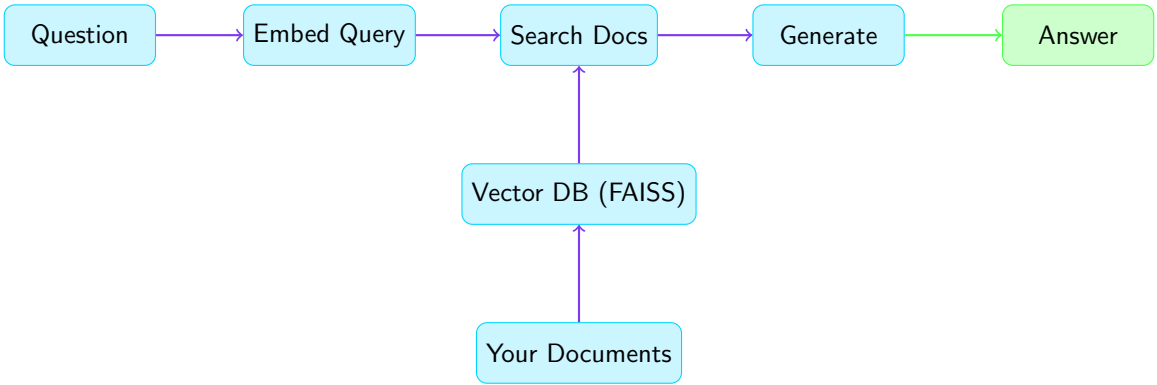
**LLM:** "I don't have information about your specific company policies..."

**With RAG**

**User:** "What is our company's leave policy?"

**RAG:** "Your company offers 25 days annual leave per year..."

# How RAG Works



# Key Components

| Component | Purpose | Tool |
|---|---|---|
| Document Loader | Load your files | LangChain |
| Text Splitter | Break into chunks | LangChain |
| Embeddings | Convert text to vectors | OpenAI |
| Vector Store | Store & search vectors | FAISS |
| LLM | Generate answers | OpenAI GPT |
| Chain | Connect it all | LangChain |

# What are Embeddings?

**Embeddings**

```
"King"  -> [0.2, 0.8, 0.1, ...]
"Queen" -> [0.3, 0.7, 0.1, ...]  <- Similar vectors!
"Car"   -> [0.9, 0.1, 0.5, ...]  <- Different vector
```

Similar concepts have similar vectors = **Semantic Search**

# What is FAISS?

**FAISS**

Facebook AI Similarity Search

- Open-source library from Meta
- Efficiently stores millions of vectors
- Finds similar vectors incredibly fast
- Free to use (no API costs!)

# The RAG Pipeline in 5 Steps

1. **LOAD** — Get your documents
2. **SPLIT** — Break into chunks
3. **EMBED** — Convert to vectors
4. **STORE** — Save in FAISS
5. **QUERY** — Retrieve + Generate

# Code Demo: Simple RAG Pipeline

```python
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import CharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS

# 1. LOAD documents
loader = TextLoader("company_policies.txt")
documents = loader.load()

# 2. SPLIT into chunks
splitter = CharacterTextSplitter(chunk_size=500)
chunks = splitter.split_documents(documents)

# 3. EMBED and STORE in FAISS
embeddings = OpenAIEmbeddings()
vector_store = FAISS.from_documents(chunks, embeddings)

# 4. QUERY - retrieve relevant chunks
retriever = vector_store.as_retriever()
docs = retriever.get_relevant_documents("annual leave?")
```

# Demo Results

Test Output

**Q: What is the annual leave policy?**

A: All full-time employees are entitled to 25 days of annual leave per calendar year. Part-time employees receive a pro-rata entitlement...

**Q: What are the core working hours?**

A: The core working hours are 10:00 AM to 4:00 PM.

# Use Cases

- **Customer Support**
  Answer questions from knowledge base
- **Legal**
  Search through contracts
- **HR**
  Query company procedures
- **Research**
  Explore academic papers
- **Documentation**
  Chat with your codebase
- **Healthcare**
  Medical knowledge retrieval

# Key Takeaways

- ✓ **RAG** = Retrieval + Generation
- ✓ Solves LLM limitations (hallucination, knowledge cutoff)
- ✓ **LangChain** makes it easy to build
- ✓ **FAISS** provides free, fast vector search
- ✓ You can build one in about 15 lines of code!

**Try the live demo:** Your Netlify URL here

# Questions?

## Q&A

**30 minutes**

**Resources:**

- LangChain: `langchain.com`
- FAISS: `github.com/facebookresearch/faiss`
- OpenAI: `platform.openai.com`

# Thank You!

Mondweep Chakravorty

github.com/mondweep/vibe-cast